



@qa_insights

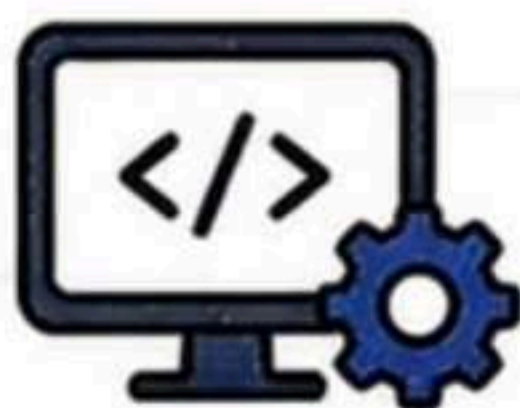
ANSIBLE

COMPLETE NOTES

(Ansible)



ANSIBLE



AUTOMATE
EVERYTHING



CONFIGURATION
MANAGEMENT



APPLICATION
DEPLOYMENT



SIMPLE
POWERFUL

BEGINNER TO ADVANCED

SWIPE TO NEXT



FOLLOW



SHARE



COMMENT



LIKE

Introduction to Ansible

① What is Ansible?



Ansible is an open-source IT automation tool that helps you configure, deploy and manage systems easily.

It uses **SSH** to communicate and does not require any agent on managed nodes.

② Why use Ansible?

- ✓ Simple & easy **YAML** syntax
- ✓ **Agentless** - no software to install
- ✓ Consistent & repeatable automation
- ✓ **Idempotent** - safe to run multiple times
- ✓ Fast and lightweight
- ✓ Large community & enterprise ready

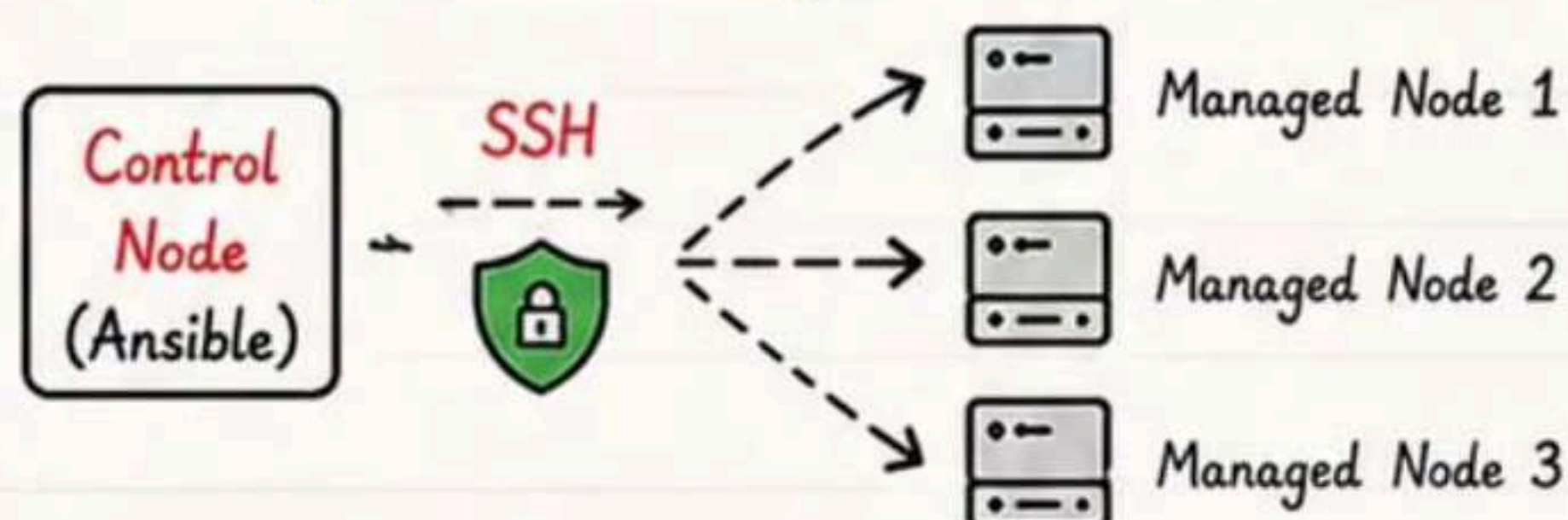
③ Key Features



- **Agentless** architecture (uses **SSH**)
- Push-based - no agents on nodes
- **YAML** based **Playbooks**
- **Idempotent** & reliable
- Strong **module** support
- Powerful automation with minimal setup
- Extensible with **Roles**, **Collections** & **Plugins**

④ Agentless Architecture

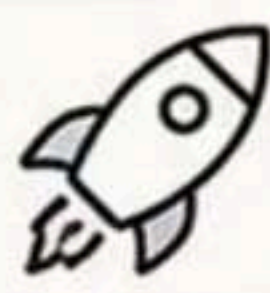
Ansible connects to remote nodes using **SSH**. No agents or additional software are required on managed machines.



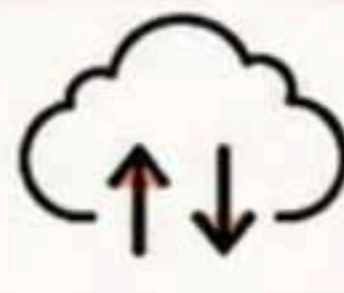
⑤ Real-world Use Cases



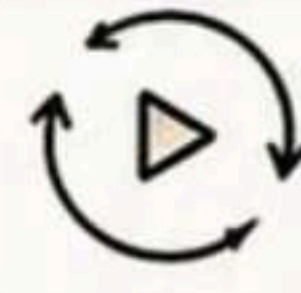
Configuration Management



Application Deployment



Infrastructure Provisioning



Continuous Integration/Delivery

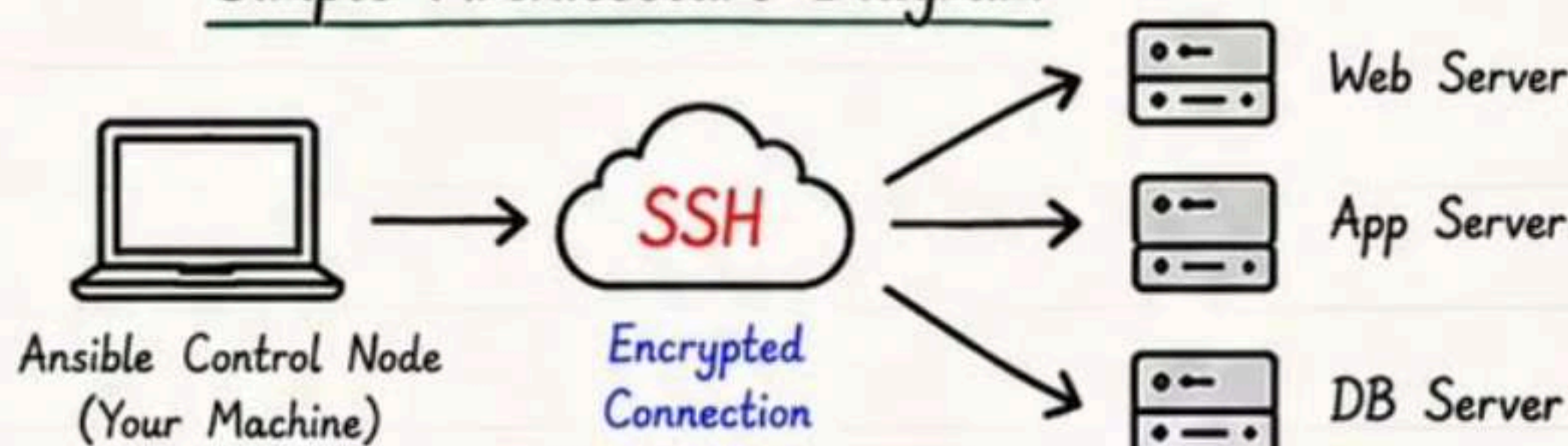


Monitoring & Maintenance

★ Interview Tip

Remember: **Ansible** is agentless, uses **SSH**, and works on a push-based model using **modules** and **playbooks**.

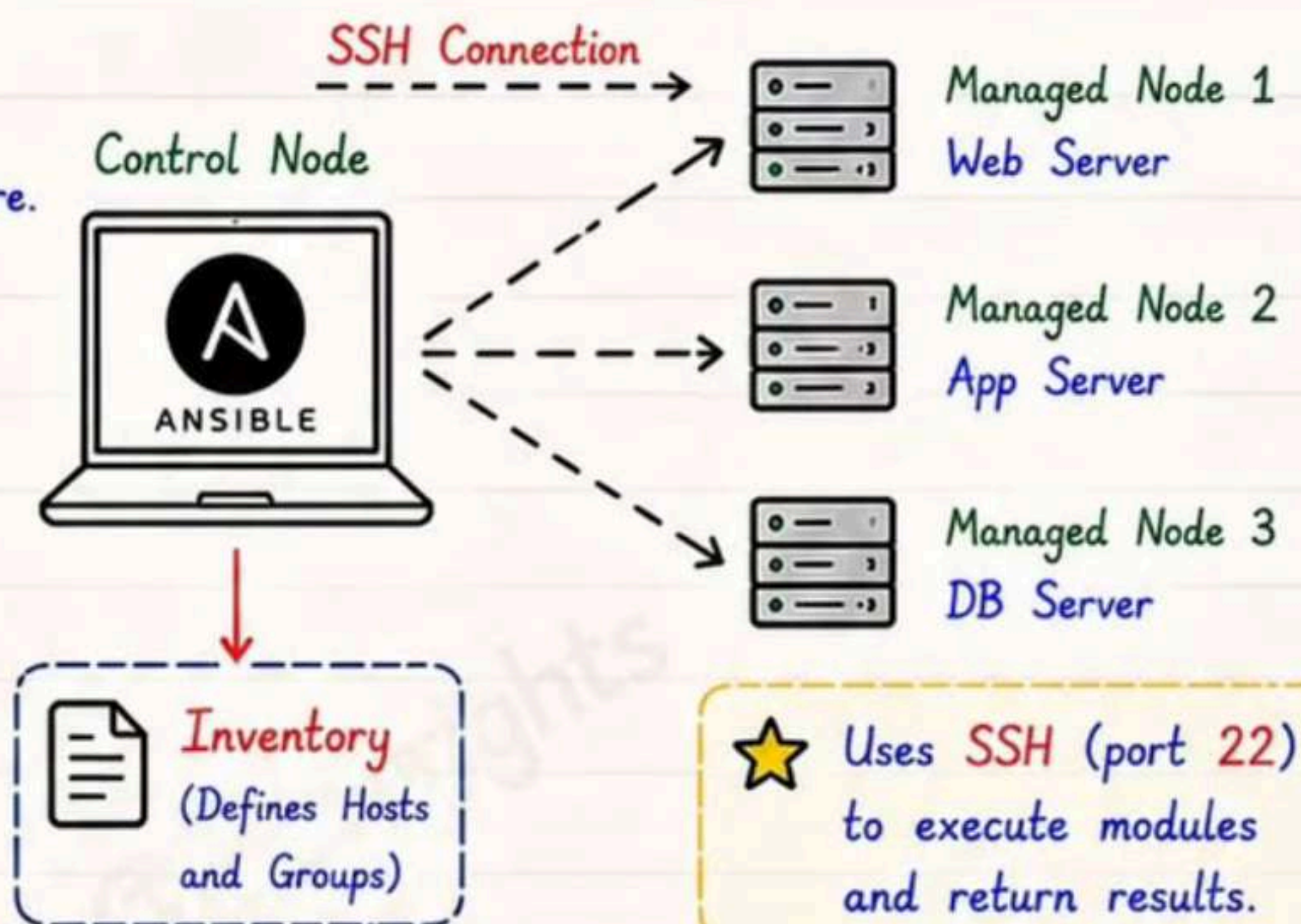
Simple Architecture Diagram



Ansible Architecture & Installation

① Ansible Architecture

- Ansible follows **AGENTLESS** architecture.
- A single **Control Node** manages one or more **Managed Nodes** over **SSH**.
- No agents or additional software required on managed nodes.



② Key Components

- ✓ **Control Node** : Machine where Ansible is installed.
- ✓ **Managed Nodes** : Target systems managed by Ansible.
- ✓ **Inventory** : List of hosts and groups.
- ✓ **SSH** : Secure communication channel.
- ✓ **Modules** : Execute tasks on managed nodes.

③ Installation (Linux)

```
# Update system
sudo apt update -y
# Install Ansible
sudo apt install ansible -y
# Verify Installation
ansible --version
```

④ Verify Installation

```
ansible --version
ansible [core 2.16.x]
config file = /etc/ansible/ansible.cfg
configured module search path =
/usr/share/ansible/plugins/modules
python version = 3.x.x
```



⑤ First Command

```
# Ping all hosts from inventory
ansible all -m ping
→ Expected Output:
server1 | SUCCESS => {"ping": "pong"}
server2 | SUCCESS => {"ping": "pong"}
```



★ Real-world Use Case

- Update packages, deploy applications, configure servers, manage users, monitor services - all from one place!



Interview Tip

Ansible is **agentless**, uses **SSH**, **YAML** Playbooks, and is **idempotent**. These are key points in interviews!

Inventory & Ad-Hoc Commands

① Inventory File



Inventory is a file that lists the hosts or nodes on which **Ansible** will run. It can be static (INI/YAML) or dynamic (cloud plugins, APIs).

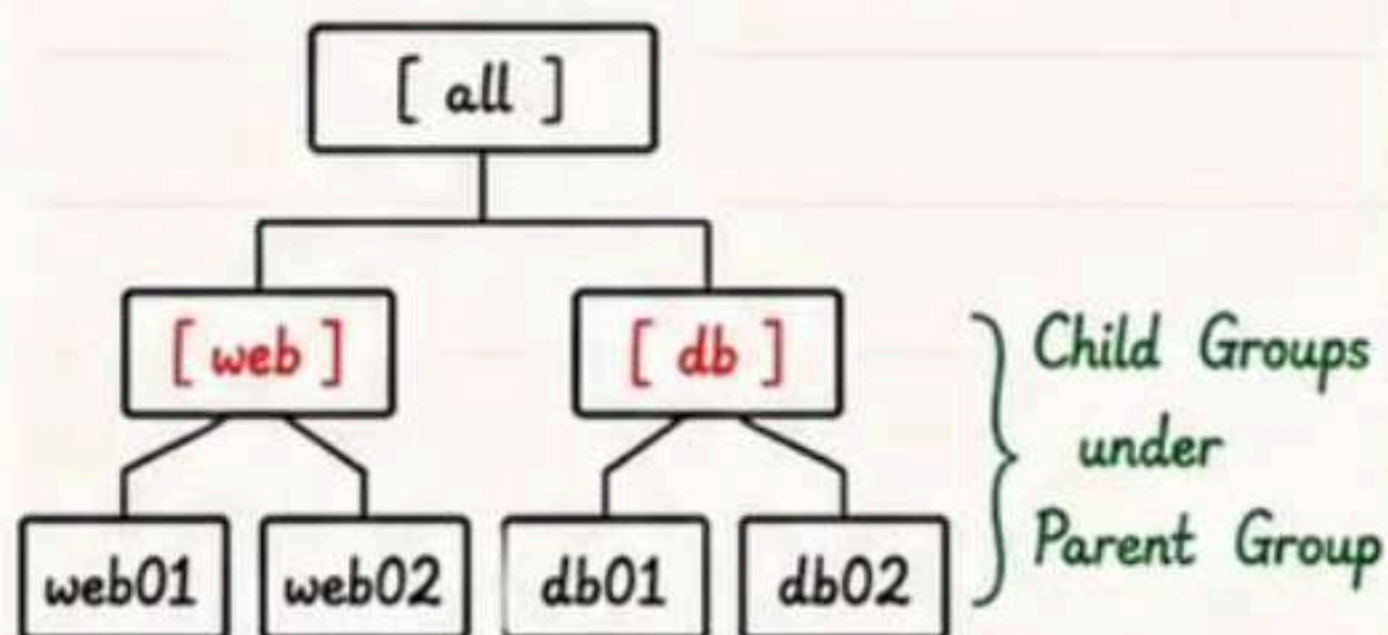
② Static vs Dynamic Inventory

- ✓ **Static Inventory:** Manually created file (INI or YAML).
- ✓ **Dynamic Inventory:** Generated automatically using scripts or plugins (AWS, Azure, GCP, etc).



③ Host Groups

Organize hosts into groups for easy management and bulk operations. Groups can have child groups.



④ INI vs YAML Inventory

INI Format (Traditional)

```

[web]
web01 ansible_host=10.0.0.1
web02 ansible_host=10.0.0.2

[db]
db01 ansible_host=10.0.1.1
db02 ansible_host=10.0.1.2
  
```

YAML Format (Modern)

```

all:
  children:
    web:
      hosts:
        web01: { ansible_host: 10.0.0.1 }
        web02: { ansible_host: 10.0.0.2 }
    db:
      hosts:
        web01: { ansible_host: 10.0.1.1 }
        db02: { ansible_host: 10.0.1.2 }
  
```

⑤ Common Ad-Hoc Commands

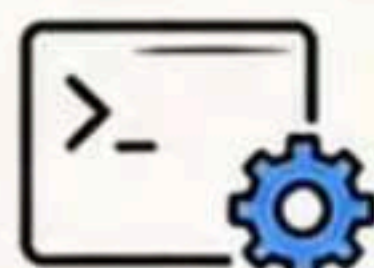
ping



Check connectivity to hosts.

```
ansible all -m ping
```

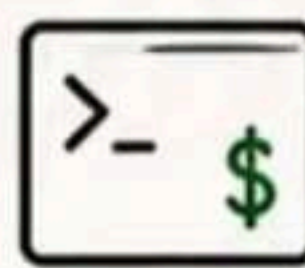
command



Run command (no shell features).

```
ansible web -m command
-a "uptime"
```

shell



Run command using shell.

```
ansible web -m shell
-a "df -h"
```

copy



Copy files from control node.

```
ansible web -m copy
-a "src=/tmp/a.txt
  dest=/tmp/a.txt"
```

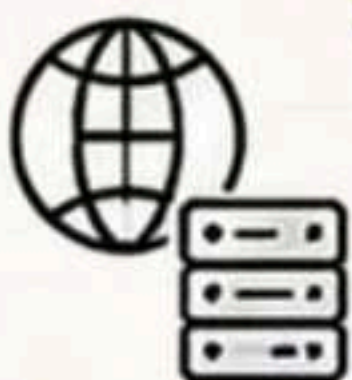
service



Manage services (start/stop/restart).

```
ansible web -m service
-a "name=nginx
  state=started"
```

★ Real-world Use Case



Use inventory groups to deploy a web app on web servers and database on db servers with a single playbook or ad-hoc command.



Interview Tip

Ad-hoc commands are great for quick tasks, but **Playbooks** are preferred for repeatable and **idempotent** automation.

Playbooks & YAML Basics

① What is a Playbook?

A **Playbook** is an **Ansible** automation file written in **YAML**. It defines what to do, on which hosts, and in what order (tasks).



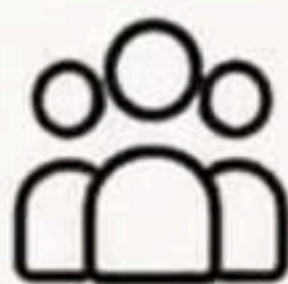
② YAML Syntax Basics

- ✓ Indentation matters (use 2 spaces)
- ✓ Use "-" for list items
- ✓ Key: Value format
- ✓ Case sensitive
- ✓ Comments start with #



YAML is human readable and easy to maintain.

③ hosts



Specifies the target hosts or host groups from the **Inventory** where the playbook will run.

④ tasks



A list of steps (tasks) executed sequentially on the target hosts using **Modules**.

⑤ become



Used to perform tasks with elevated privileges (like **sudo/root**).

⑥ Simple Playbook Example

```

---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx package
      yum:
        name: nginx
        state: present
    - name: Start Nginx service
      service:
        name: nginx
        state: started
        enabled: yes
  
```

Annotations for the example:

- Play name (description) points to `name: Install and start Nginx`
- Target hosts /groups from Inventory points to `hosts: webservers`
- Run tasks with sudo privileges points to `become: yes`
- List of tasks to execute points to the `tasks:` section
- Install Nginx if not present points to `name: nginx` and `state: present`
- Start service and enable it points to `name: nginx`, `state: started`, and `enabled: yes`

How a Playbook Works



★ Real-world Use Case

Deploy **Nginx** on multiple web servers, configure it, and ensure the service is always running using a **Playbook**.

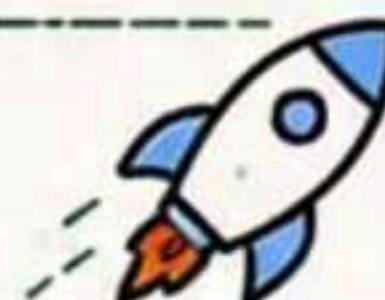
💡 Interview Tip

Playbooks bring repeatability, **idempotency** and consistency to infrastructure automation.



Key Takeaway

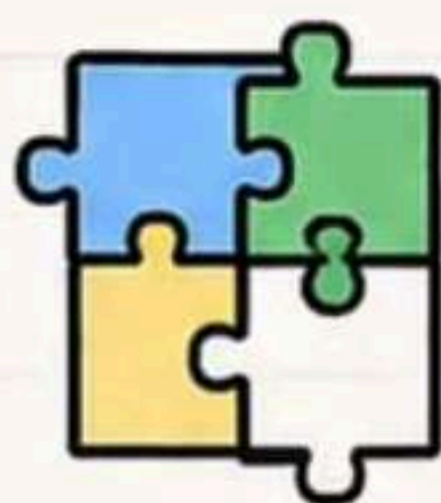
Playbooks + YAML are the heart of **Ansible Automation**. Define once, run anywhere!



Ansible Modules

① What are Modules?

Modules are reusable units of code in **Ansible** that perform specific tasks on remote hosts. They make **Playbooks** simple, powerful and **idempotent**.



② Why use Modules?

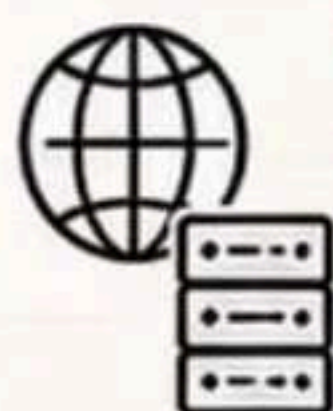
- ✓ Built-in and easy to use
- ✓ **Idempotent** by default
- ✓ Cross-platform support
- ✓ Consistent and reliable automation
- ✓ Large collection in **Ansible Galaxy**



③ Commonly Used Ansible Modules

Module	Purpose	Example Usage	Notes
 command	Runs commands without shell features.	<code>ansible web -m command -a "uptime"</code>	✓ Use for simple commands. More secure.
 shell	Runs commands using shell.	<code>ansible web -m shell -a "df -h"</code>	✓ Use when you need shell features (pipes, redirects).
 copy	Copies files from control node to remote host.	<code>ansible web -m copy -a "src=/tmp/app.conf dest=/etc/app.conf"</code>	✓ Best for small files. For big files use synchronize module.
 file	Manages files, directories, permissions and states.	<code>ansible web -m file -a "path=/opt/app state=directory mode=755"</code>	✓ Create/delete files or dirs, set permissions.
 service	Manages services (start/stop/restart).	<code>ansible web -m service -a "name=nginx state=started enabled=yes"</code>	✓ Works on most Linux distributions.
 package	Installs, removes or manages packages.	<code>ansible web -m package -a "name=nginx state=present"</code>	✓ OS independent. Uses underlying package mgr.
 user	Manages user accounts and attributes.	<code>ansible web -m user -a "name=devops state=present shell=/bin/bash groups=sudo"</code>	✓ Create, update or remove users and groups.
 apt / yum	Package management for Debian/RHEL based systems.	<code>ansible web -m apt -a "name=git state=present"</code> <code>ansible web -m yum -a "name=git state=present"</code>	✓ Use apt for Debian/Ubuntu, yum/dnf for RHEL/CentOS.

④ Real-world Use Case



Using modules, you can deploy an application end-to-end — create users, install packages, copy configs, start services and ensure everything is in the desired state (**Idempotent**).



Interview Tip

Modules are the heart of **Ansible**. Know the purpose of each module and when to use it.



Variables & Facts

① Variables in Ansible

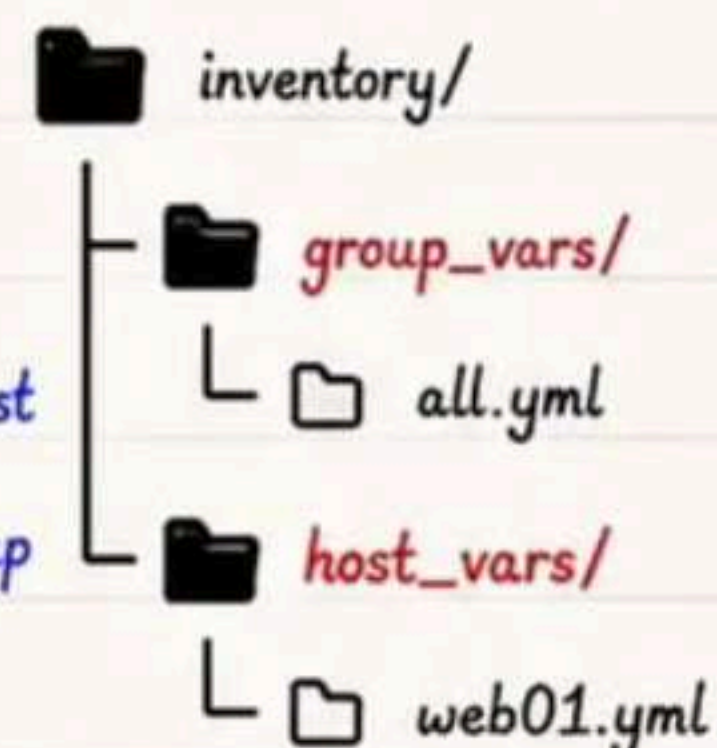
Variables store data that can be reused in **Playbooks**. They make Playbooks flexible and dynamic.

```
x = 10
name = web
env = dev
```

Variables

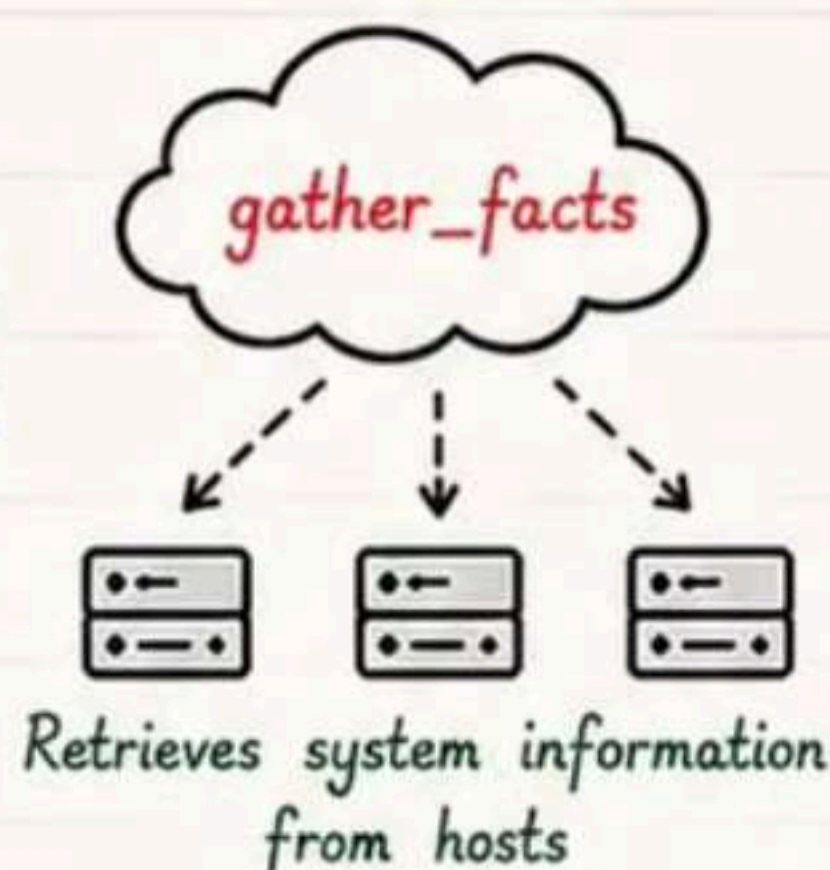
② Types of Variables

- ✓ **vars**: Defined in Playbook
- ✓ **host_vars**: Variables for specific Host
- ✓ **group_vars**: Variables for Host Group
- ✓ **extra_vars**: Passed from CLI
- ✓ **facts**: Automatically discovered data



③ Facts in Ansible

Facts are system information collected from managed hosts. They are stored in **ansible_facts** variable.



Common **ansible_facts**

- ✓ **ansible_hostname**
- ✓ **ansible_os_family**
- ✓ **ansible_distribution**
- ✓ **ansible_ip_addresses**
- ✓ **ansible_memory_mb**
- ✓ **ansible_processor_vcpus** and many more...

④ Example Usage

```
- hosts: webservers
  gather_facts: yes
  vars:
```

```
    app_name: Nginx
    port: 80
```

```
  tasks:
```

```
- name: Show OS Family
```

```
  debug:
```

```
    msg: "OS Family is {{ ansible_facts.os_family }}"
```

```
- name: Install {{ app_name }}
```

```
  package:
```

```
    name: nginx
```

```
    state: present
```

Using Variable

```
{{ app_name }}
```

Using Facts

```
{{ ansible_facts.os_family }}
```

⑤ Variable Precedence (High → Low)

- 1 extra_vars (CLI)
- 2 task vars
- 3 block vars
- 4 role vars (and defaults)
- 5 play vars
- 6 host_vars (host level)
- 7 group_vars (group level)
- 8 inventory vars
- 9 facts (setup/gathered facts)
- 10 role defaults

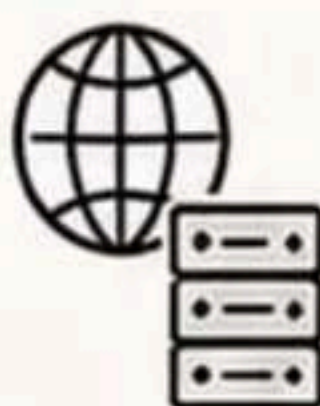
Higher Priority

Lower Priority



Interview Tip

Facts are dynamic and collected from hosts at runtime, while **Variables** are defined by us.



Real-world Use Case

Use **variables** for configuration values (ports, paths, package names) and **facts** for making decisions based on server information (OS, memory, interfaces, etc.).

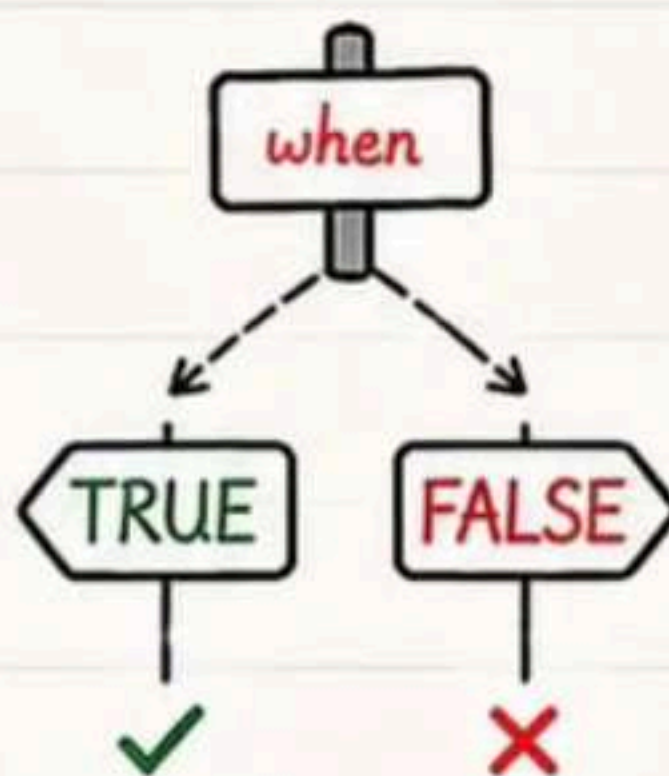
Conditionals & Loops

① Conditionals (when)

Use **when** to run tasks only when a condition is true.

Common Tests

- ✓ **is defined** → variable exists
- ✓ **is not defined** → variable does not exist
- ✓ **==, !=** → equality check
- ✓ **>, <, >=, <=** → numeric comparison
- ✓ **in, not in** → membership check



② is defined / is not defined

- **is defined** → Check if variable is available
- **is not defined** → Check if variable is missing

★ Interview Tip

is defined and **is not defined** are commonly asked in Ansible interviews.

③ Loops

Use **loop** to repeat a task for a list of items.

Loop Examples

loop, with_items, loop_control
(loop is the modern way)

④ Register Output

Use **register** to capture the output of a task and use it in next tasks.

Registered Data

Stored as a variable and can be accessed using **.stdout**, **.rc**, **.stdout_lines**, **.results**, etc.

⑤ Accessing Registered Data

- **result.stdout** → Output
- **result.stdout_lines** → List of lines
- **result.rc** → Return code
- **result.results** → For loop items

⑥ Practical Examples

1. when + is defined

```

- name: Install Nginx if pkg_name is defined
  apt:
    name: "{{ pkg_name }}"
    state: present
  when: pkg_name is defined
  
```

Use Case:

Install a package only when the variable is provided.



2. Loop Example

```

- name: Create multiple users
  user:
    name: "{{ item }}"
    state: present
  loop:
    - devops
    - qa
    - support
  
```

Use Case:

Create multiple user accounts in one go.



3. Register Example

```

- name: Check disk usage
  shell: df -h
  register: disk_result

- name: Show disk output
  debug:
    var: disk_result.stdout_lines
  
```

Use Case:

Capture command output and use it in next tasks.



★ Real-world Use Case



Playbook



Ansible
Automation



Managed Hosts

- ✓ Run tasks only when needed
- ✓ Repeat actions across servers
- ✓ Use outputs for decisions
- ✓ Idempotent & reliable



Best Practice

- ✓ Prefer **loop** over **with_items**
- ✓ Use **register** to avoid re-running tasks
- ✓ Combine **when** with facts/variables
- ✓ Keep conditions simple & readable

Handlers & Tags

① Handlers (What & Why?)

Handlers are special tasks that run only when notified by other tasks. They are executed at the end of a Play.



Handler

Key Points

- ✓ Run **once**, even if notified multiple times
- ✓ Executed **after all tasks** in the Play
- ✓ Useful for restarting services, reloading configs, etc.

② Handlers Example

```
- name: Update Nginx Config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    notify: Restart Nginx
...
handlers:
  - name: Restart Nginx
    service:
      name: nginx
      state: restarted
```

This handler runs only if the template task makes a change.

③ Tags

Tags help you run or skip specific parts of a Playbook.



Add tags to tasks

```
- name: Install Nginx
  apt:
    name: nginx
    state: present
  tags:
    - install
    - web
```

Run with tags

```
ansible-playbook site.yml --tags "install"
# Run only tasks with tag 'install'

ansible-playbook site.yml --skip-tags "web"
# Skip tasks with tag 'web'

ansible-playbook site.yml --tags "install,web"
# Run multiple tags
```

④ skip-tags

Use **--skip-tags** to exclude tasks with specific tags.

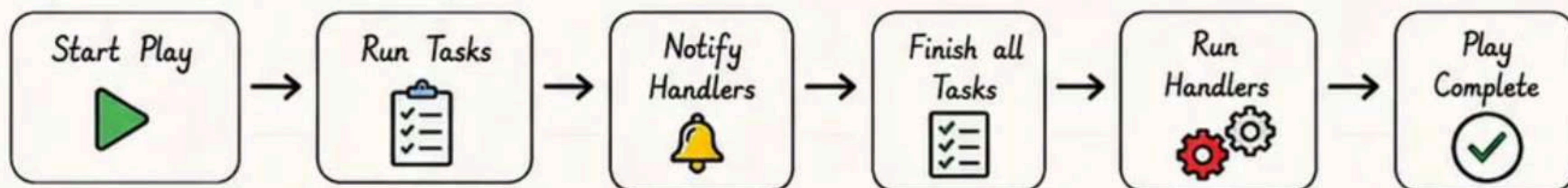
Example

```
ansible-playbook site.yml --skip-tags "debug,test"
# All tasks will run except those
  tagged as 'debug' or 'test'
```

★ Best Practice

Always tag long tasks (**install**, **backup**, **db_migrate**) so you can run or skip them easily.

⑤ Execution Flow (Play + Handlers)



How it works?

- Tasks run in order.
- If a task notifies a handler, it is marked.
- After all tasks finish, only notified handlers run.
- Handlers run once, even if notified multiple times.



Real-world Use Case

When you update a config file, you **notify** a handler to restart the service. This ensures changes are applied without unnecessary restarts (**Idempotent & Efficient**).



Interview Tip

Handlers run at the end of the Play, only if notified. Tags help in selective execution, making Playbooks flexible and powerful.



Ansible Roles

① Why Roles?

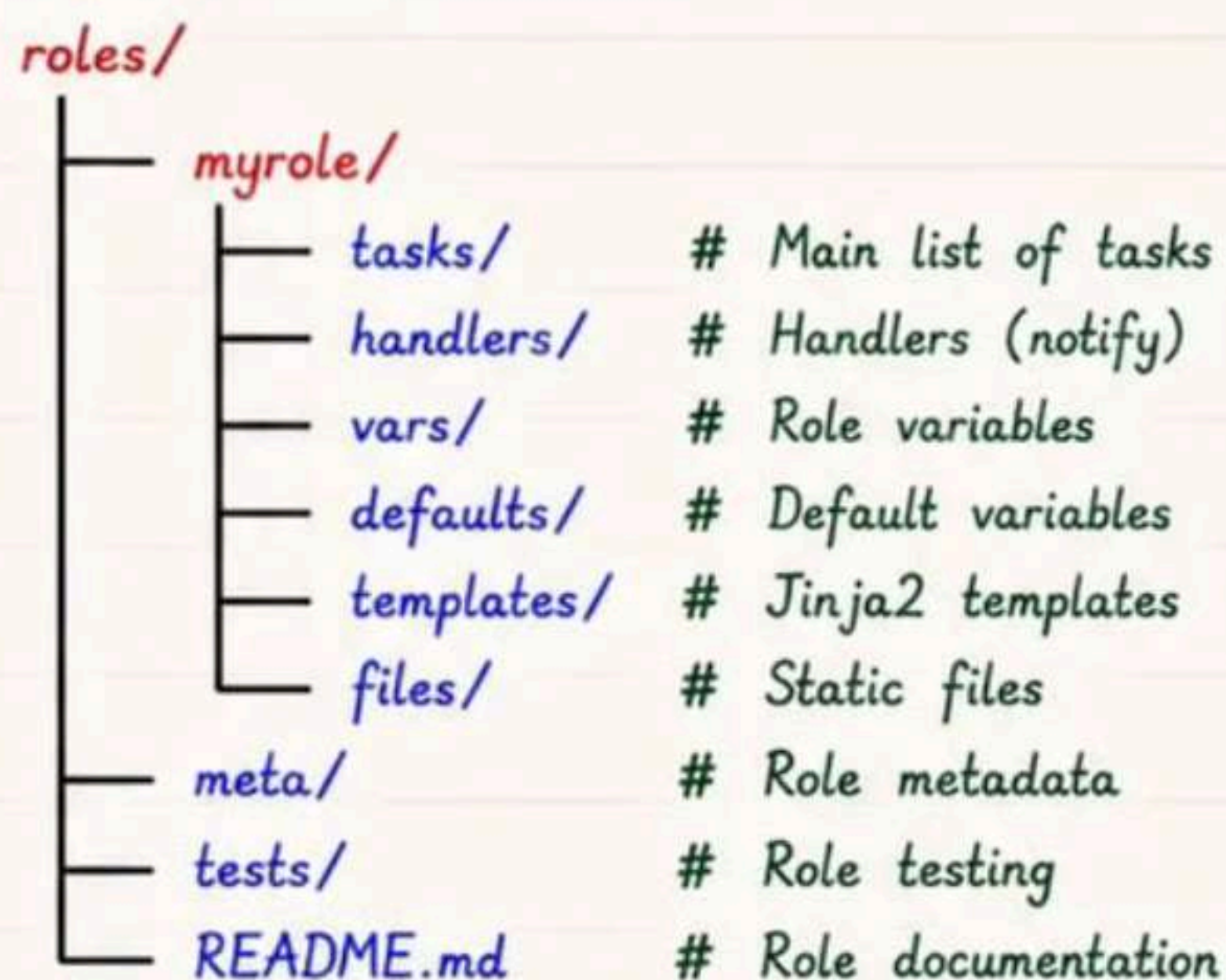
Roles help you organize and reuse Ansible content in a standard structure. They make **Playbooks** clean, modular and easy to maintain.

Benefits

- ✓ Reusability
- ✓ Standardized structure
- ✓ Easier collaboration
- ✓ Separation of concerns
- ✓ Publish & share via **Galaxy**



② Directory Structure



③ Role Directories Explained

Directory	Purpose	Example
 tasks/	Define the main tasks executed by the role.	# roles/myrole/tasks/main.yml - name: Install Nginx apt: name=nginx state=present
 handlers/	Define handlers that run when notified.	# roles/myrole/handlers/main.yml - name: Restart Nginx service: name=nginx state=restarted
 vars/	Variables with higher precedence (role level).	# roles/myrole/vars/main.yml nginx_port: 80
 defaults/	Default variables with low precedence (overridable).	# roles/myrole/defaults/main.yml nginx_port: 80
 templates/	Jinja2 templates for dynamic configuration.	# roles/myrole/templates/nginx.conf.j2 server_name {{ server_name }};
 files/	Static files copied as-is to managed hosts.	# roles/myrole/files/motd.txt (Hello from Ansible!)

★ Interview Tip

Roles enforce best practices and make **Automation** scalable. Know the directory structure and variable precedence while using roles.



Real-world Use Case

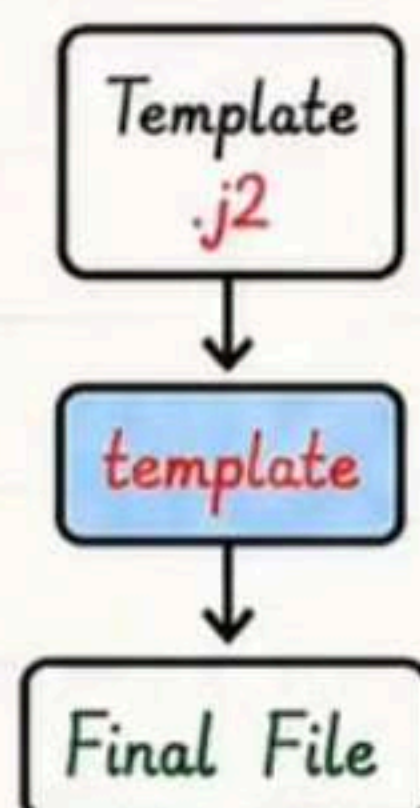
Use **roles** to automate common components like **Nginx**, **MySQL**, **Docker**, and **Users** across multiple environments (Dev, QA, Prod).

Templates & Jinja2

① What are Templates?

Templates allow dynamic configuration files to be generated using Jinja2 syntax and variables.

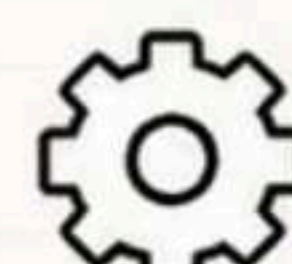
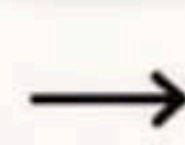
The `template` module replaces variables and renders the final file on the managed `Host`.



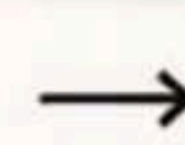
② How it works?



Template
(.j2 file)



Jinja2
Engine



Rendered
File

Ansible reads the template, replaces variables using Jinja2 and generates the final configuration on the target `Host`.

③ Variables in Templates

Use `{{ variable_name }}` to insert variables.

Example: `nginx.conf.j2`

```
server {
    listen {{ nginx_port }};
    server_name {{ domain_name }};
    root {{ doc_root }};
}
```

④ Filters in Jinja2

Filters help to transform or format data.

- `default()` → Default value
- `upper / lower` → Change case
- `trim` → Remove spaces
- `int / float` → Type conversion
- `join` → Join list items

Example:

```
{{ app_name | upper }} # Output: APP
```

⑤ Loops in Templates

Use for loop to iterate over lists.

Example: `users.txt.j2`

```
{% for user in users %}
    {{ user.name }}:{{ user.uid }}
{% endfor %}
```

★ Interview Tip

Jinja2 is the heart of `Ansible` Templates. Know variables, filters and loops well!



⑥ Conditionals in Templates

Use `if`, `elif`, `else` for conditional logic.

```
{% if environment == 'prod' %}
    server_name prod.example.com;
{% elif environment == 'dev' %}
    server_name dev.example.com;
{% else %}
    server_name test.example.com;
{% endif %}
```



Real-world Use Case

Generate configuration files for Nginx, Apache, MySQL, Docker, Kubernetes, Application configs, and environment-specific settings across `Dev`, `QA`, `Prod`.

⑦ Practical Example (Playbook)

```
- name: Generate Nginx configuration from Template
  hosts: webservers
  become: yes
  vars:
    nginx_port: 80
    domain_name: example.com
    doc_root: /var/www/html
  tasks:
    - name: Deploy Nginx config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
```

Template: `nginx.conf.j2`

```
server {
    listen {{ nginx_port }};
    server_name {{ domain_name }};
    root {{ doc_root }};
}
```



Generated File on Host

```
server {
    listen 80;
    server_name example.com;
    root /var/www/html;
}
```

Best Practices

- ✓ Keep template files in `templates/` directory.
- ✓ Use meaningful variable names.

- ✓ Validate templates before deploying.
- ✓ Avoid complex logic in templates.

- ✓ Use filters to keep templates clean.
- ✓ Test with `--check` before running.

Ansible Vault

① Why Vault?

Ansible Vault helps you encrypt sensitive data like passwords, API keys, tokens, certificates, etc. before storing in your Playbooks or variable files.

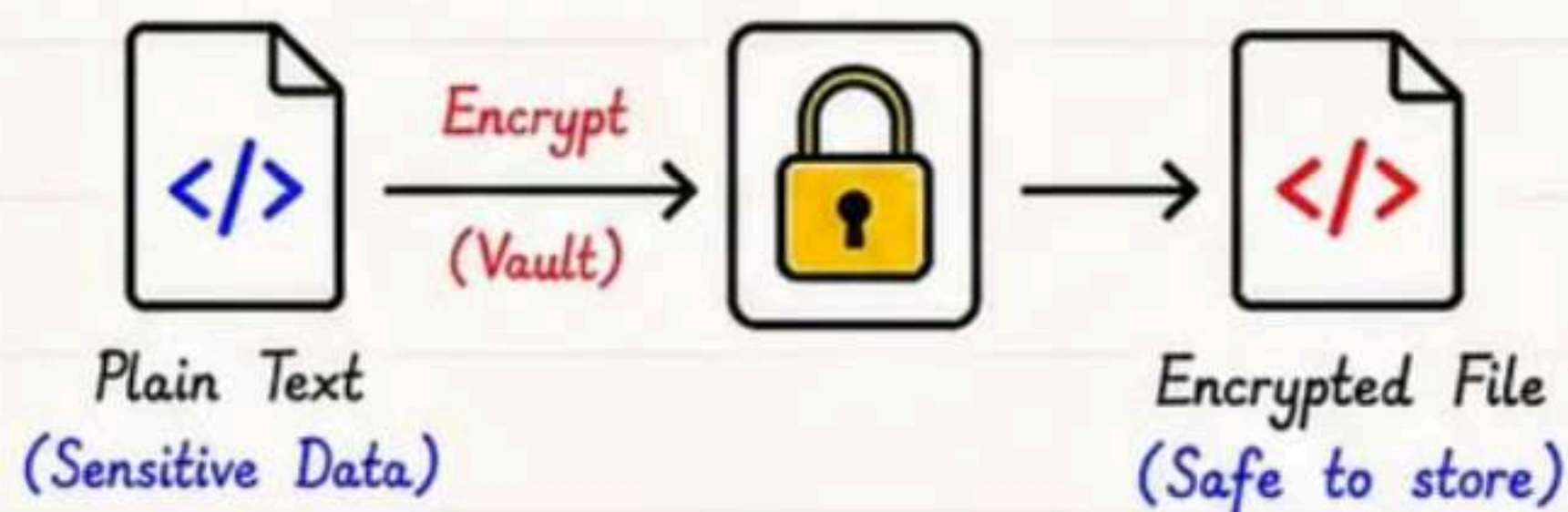


Vault

Benefits

- ✓ Protects sensitive information
- ✓ Safe to store in Git repositories
- ✓ Easy to use with Playbooks
- ✓ Supports multiple encryption formats

② How Vault Works?



★ Interview Tip

Always encrypt sensitive data before committing to Git. Never store plain text secrets in Playbooks.

③ Vault Commands

Encrypt a File



`ansible-vault encrypt file.yml`

Example:

Encrypt a variable file.
→ Prompts for Vault password.

Decrypt a File



`ansible-vault decrypt file.yml`

Example:

Decrypt and view content in plain text.

Edit a File



`ansible-vault edit file.yml`

Example:

Opens file in editor, saves and keeps it encrypted.

View a File



`ansible-vault view file.yml`

Example:

View encrypted file without editing.

Rekey a File



`ansible-vault rekey file.yml`

Example:

Change Vault password for an existing file.

④ Password File (Best Practice)

Store your Vault password in a file and provide it while running commands.



`vault_pass.txt`

Use Password File

`ansible-vault --vault-password-file vault_pass.txt <command>`

Example:

```
ansible-vault --vault-password-file vault_pass.txt encrypt file.yml # Encrypt
ansible-vault --vault-password-file vault_pass.txt decrypt file.yml # Decrypt
ansible-vault --vault-password-file vault_pass.txt edit file.yml # Edit
```

⚠ Important Note

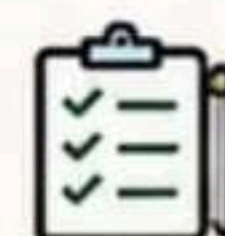
- Protect your password file with strict permissions.
- Never commit the password file to Git.
- Use .gitignore to exclude it.

```
chmod 600 vault_pass.txt
echo 'vault_pass.txt' >> .gitignore
```



⑤ Best Practices

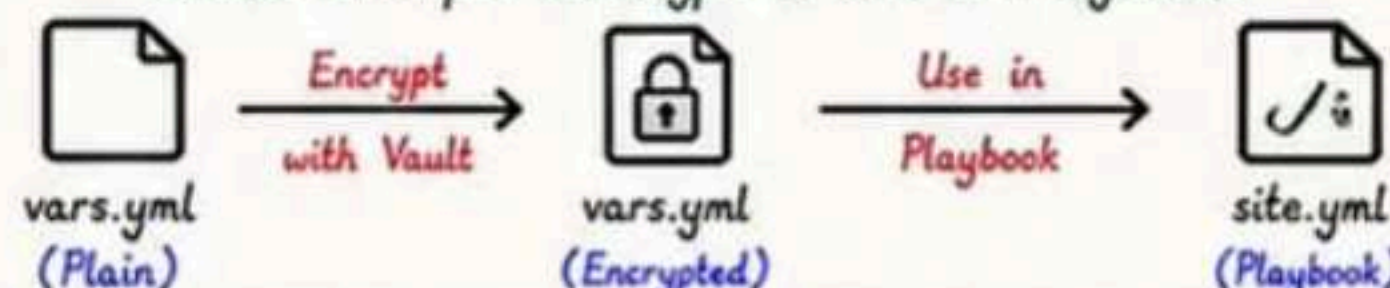
- ✓ Encrypt all sensitive data before storing in Git.
- ✓ Use password file instead of typing password repeatedly.
- ✓ Limit access to password file (chmod 600).
- ✓ Use Vault IDs for multiple passwords in enterprise setups.
- ✓ Always test Playbooks after decrypting to avoid failures.



Real-world Use Case

You can encrypt database passwords, cloud access keys, API tokens, and other secrets using Ansible Vault and safely automate deployments across Dev, QA, and Prod environments.

Quick Example (Encrypt & Use in Playbook)



Error Handling & Blocks

① ignore_errors

Ignores errors for a task and continues execution.



```
- name: Run risky command
  command: /bin/false
  ignore_errors: yes # Continue
                    even if this fails
```

② failed_when

Define custom conditions that mark a task as failed.



```
- name: Check disk space
  shell: df -h /
  register: disk
  failed_when: disk.stdout.find('100%') != -1
```

③ changed_when

Override the default changed status with a custom condition.



```
- name: Touch file
  command: /bin/touch /tmp/test.txt
  register: result
  changed_when: false # Always show
                    no change
```

⑤ Retry Logic (until, retries, delay)

Retry a task until it succeeds.
Useful for transient failures.



```
- name: Wait for service to start
  shell: systemctl is-active nginx
  register: svc
  retries: 5 # Try 5 times
  delay: 10 # Wait 10 sec between tries
  until: svc.rc == 0 # Success when rc == 0
```

④ Block / Rescue / Always

Group tasks and handle errors gracefully.



```
- block:
  - name: Do main task
    command: /bin/false
  - name: Another task
    command: echo "Success"

  rescue:
  - name: Handle failure
    debug:
      msg: "Something went wrong!"

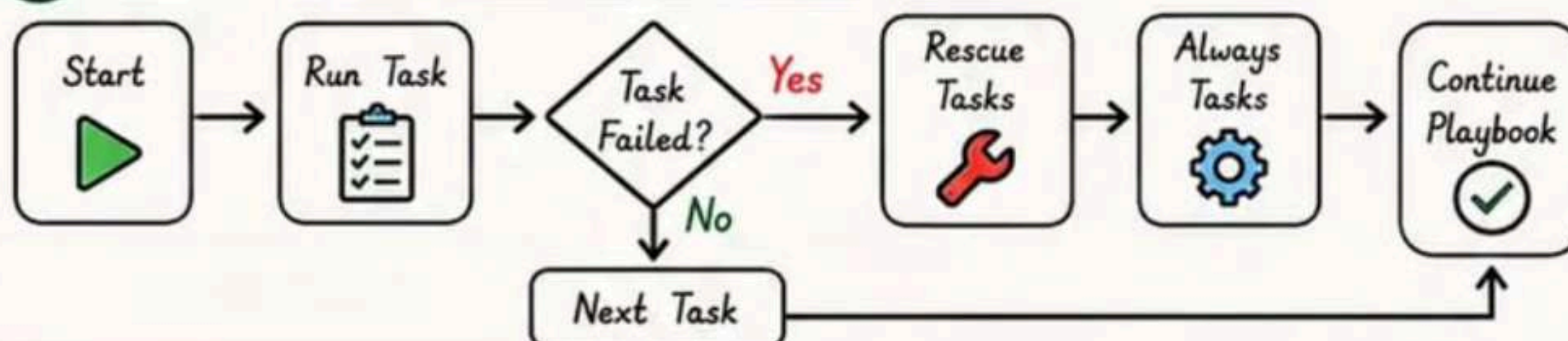
  always:
  - name: Always run this
    debug:
      msg: "This runs no matter what"
```

} Run main tasks

} Runs only if block fails

} Runs always

⑥ Execution Flow with Error Handling



★ Interview Tip

Block/Rescue/Always is similar to try/catch/finally.
Use it to make Playbooks resilient and reliable.



Best Practices

- ✓ Use failed_when for precise failure detection.
- ✓ Use changed_when to keep idempotency.
- ✓ Avoid ignore_errors unless absolutely needed.
- ✓ Use blocks to handle complex failures.
- ✓ Always provide meaningful messages in rescue.
- ✓ Use retry logic for unstable services/APIs.



Real-world Use Case

When deploying web apps, services may take time to start.
Use retries + blocks to ensure smooth and automated deployments.

Ansible Galaxy & Collections

① Galaxy

Ansible Galaxy is the official hub for sharing and discovering Ansible content like Roles and Collections.



What you get on Galaxy?

- ✓ High quality Roles & Collections
- ✓ Community and vendor content
- ✓ Versioned, tested and reusable
- ✓ Easy to integrate into Playbooks

③ Install Roles from Galaxy

Install a role

`ansible-galaxy install geerlingguy.nginx`

Install specific version

`ansible-galaxy install geerlingguy.nginx:2.2.0`

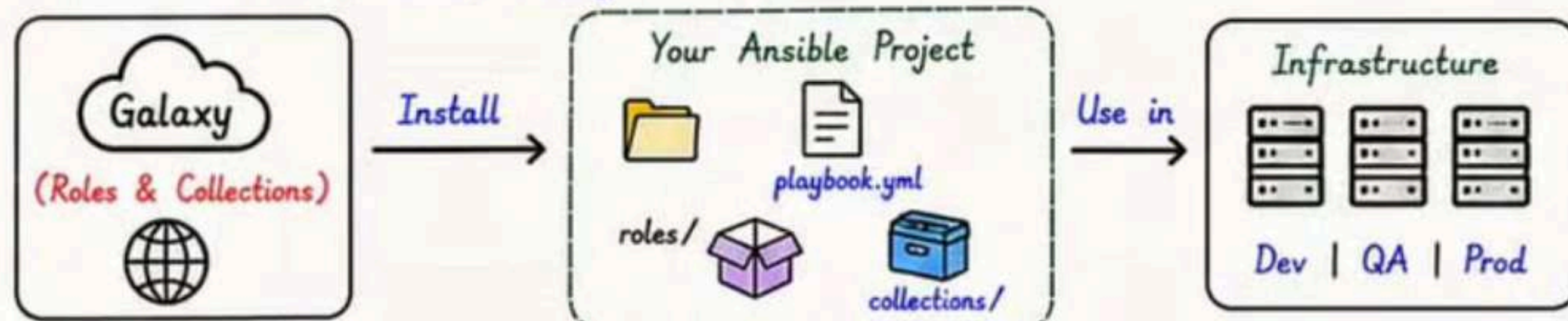


Installed Location (default)

```
~/.ansible/roles/
├── geerlingguy.nginx/
│   ├── tasks/
│   ├── defaults/
│   └── meta/ ...
```

⑤ Reusable Automation

Using Galaxy content saves time and ensures best practices. Focus on automation, not reinventing the wheel!



② Collections

Collections are the next-generation packaging format introduced in Ansible 2.10+.



Why Collections?

- ✓ Bundle of modules, plugins, roles, docs & tests
- ✓ Namespaced (e.g., community.general)
- ✓ Better versioning and dependency management
- ✓ Standardized and future-ready content

④ Install Collections

Install a collection

`ansible-galaxy collection install community.general`

Install multiple collections

`ansible-galaxy collection install \`
`community.general \`
`ansible.posix`



Installed Location (default)

```
~/.ansible/collections/ansible_collections/
├── community/
│   └── general/
├── ansible/
│   └── posix/
```

★ Interview Tip

Galaxy helps in reusability, consistency and collaboration. Collections are the future of Ansible content management.

Best Practices

- ✓ Always use trusted publishers
- ✓ Check role/collection documentation
- ✓ Pin versions for stability
- ✓ Keep Galaxy content updated
- ✓ Review code before using
- ✓ Follow least privilege and security guidelines

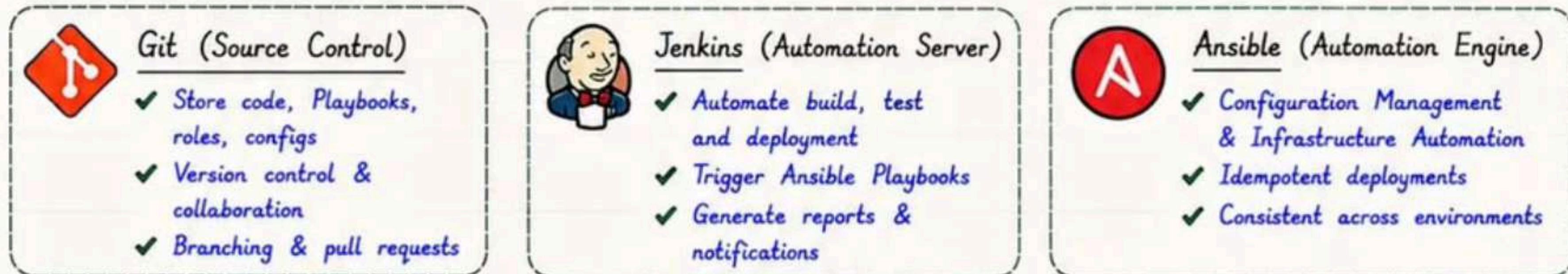


Real-world Use Case

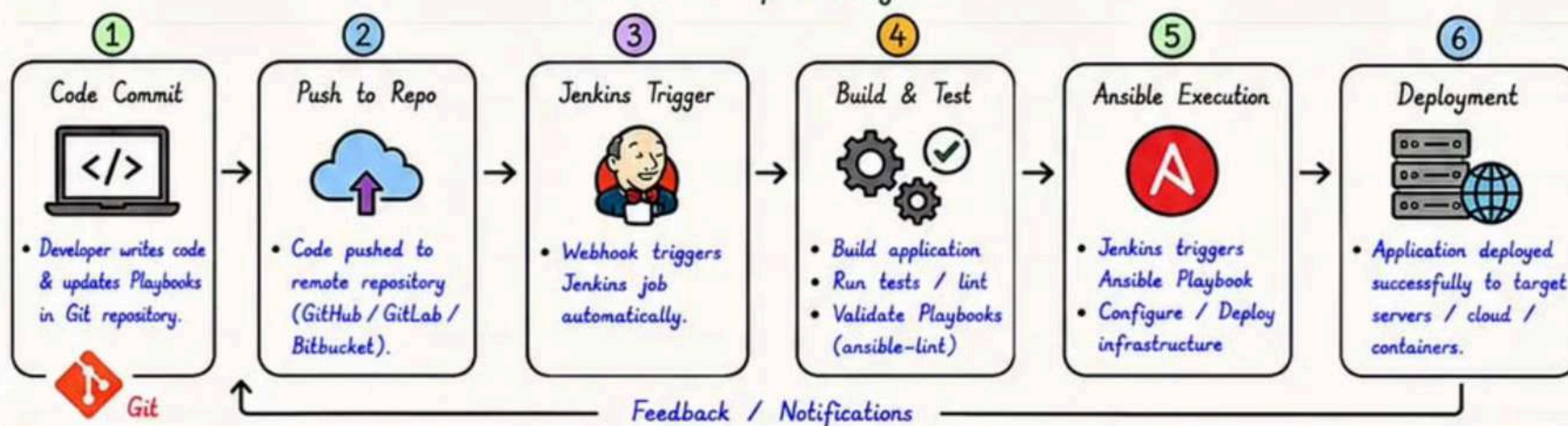
Use community.general to manage users, packages, files, services across Linux environments efficiently and consistently.

Real-World DevOps Workflow

Git + Jenkins + Ansible : Deployment Pipeline & Configuration Management



CI/CD Workflow Diagram



Configuration Management with Ansible

- ✓ Ensure systems are in desired state
- ✓ Manage users, packages, services, files, permissions
- ✓ Maintain consistency across Dev, QA, Prod
- ✓ Enforce security & compliance
- ✓ Drift detection & remediation



Infrastructure Automation



Provision → Configure → Deploy → Validate → Monitor

Ansible works across multiple platforms & environments
Agentless, SSH based, Simple & Powerful!

Example Pipeline (Jenkinsfile + Ansible)

Jenkinsfile (Simplified)

```

pipeline {
    agent any
    stages {
        stage('Checkout') { steps { git 'repo-url' } }
        stage('Build & Test') { steps { sh 'mvn clean test' } }
        stage('Ansible Deploy') {
            steps {
                sh 'ansible-playbook -i inventory deploy.yml'
            }
        }
    }
}
  
```

deploy.yml (Ansible Playbook)

```

- hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      package:
        name: nginx
        state: present
    - name: Copy App Files
      copy:
        src: app/
        dest: /var/www/html/
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
  
```



Best Practices

- ✓ Store Ansible code in Git
- ✓ Use Roles & Collections for reusability
- ✓ Keep Playbooks idempotent
- ✓ Use Jenkins Credentials securely
- ✓ Validate Playbooks (ansible-lint)
- ✓ Use Inventory groups & variables
- ✓ Implement notifications (Email/Slack)
- ✓ Monitor deployments & rollbacks



Real-world Use Cases

- Deploy web applications automatically
- Configure servers consistently
- Patch management & compliance
- Infrastructure provisioning in cloud
- Environment setup (Dev / QA / Prod)



Interview Tip

DevOps is all about Automation, Consistency & Speed.
Ansible + Jenkins + Git = Reliable CI/CD Pipelines!

Ansible Best Practices

Following Best Practices in Ansible helps in writing clean, secure, efficient, reusable and maintainable automation.

① Idempotency



- Ansible should produce the same result every time a **Playbook** is run.
- Use modules that are idempotent (e.g., **copy**, **yum**, **user**).
- Avoid using **command** or **shell** unless necessary.

★ Tip: Idempotency = Reliable Automation

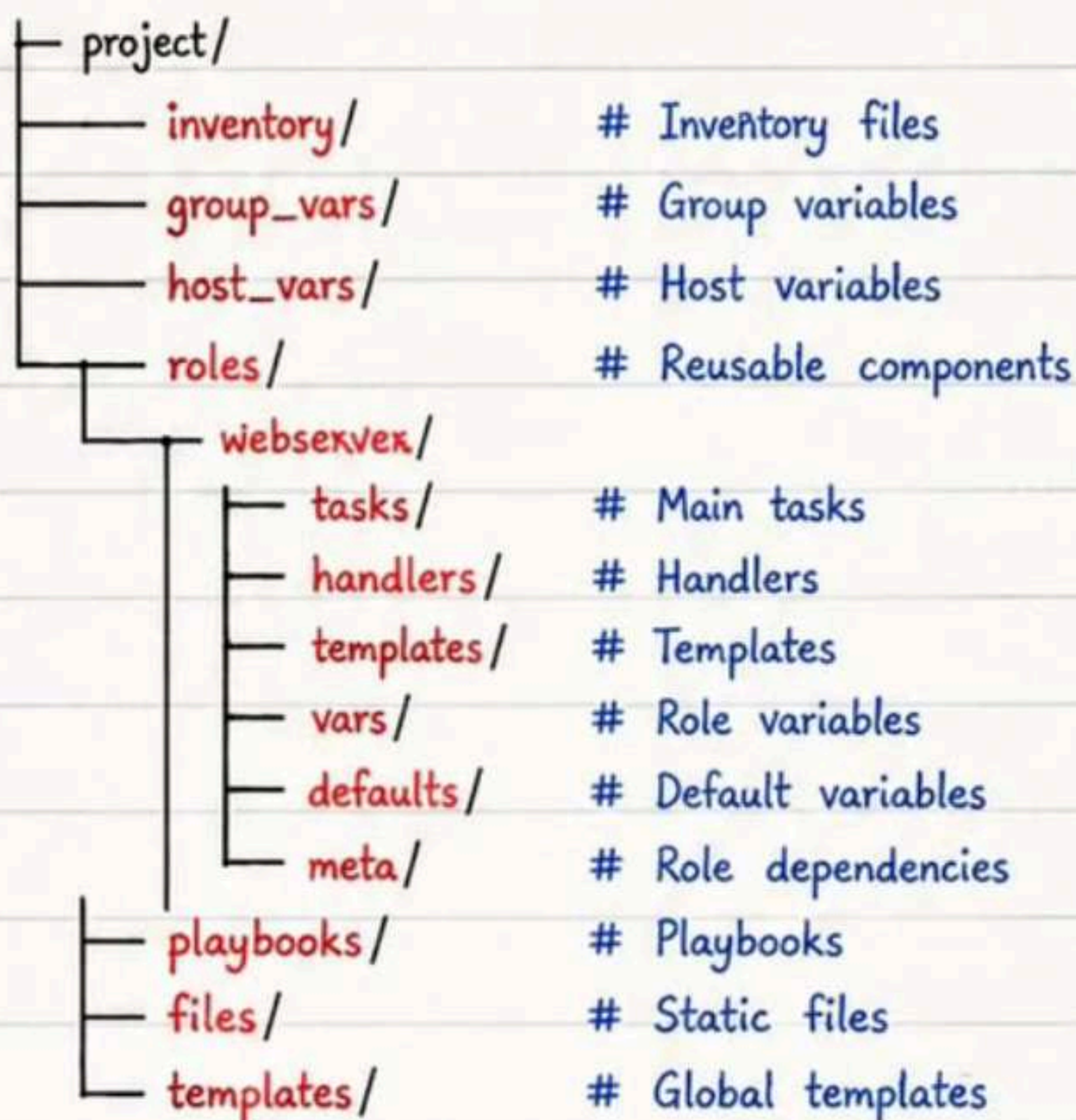
③ Naming Conventions



- Use **lowercase** and **underscores**.
- Playbooks: **site.yml**, **webserver.yml**
- Roles: **webserver**, **database**
- Variables: **user_name**, **db_password**
- Files/Templates: **httpd.conf.j2**

★ Tip: Clear names = Easy readability

② Folder Structure



④ Security

- Use **Ansible Vault** for sensitive data.
- Avoid hardcoding **credentials**.
- Use **become** for privilege escalation instead of **root** login.
- Restrict **SSH** access using keys.

💡 Remember: Security first in DevOps!

⑤ Performance Tips

- Use **SSH** pipelining (**pipelining = True**).
- Limit **facts** gathering (**gather_facts: no**).
- Use **async** + **poll** for long tasks.
- Run tasks in **parallel** (**forks**).
- Use **tags** to run only required tasks.

✓ Faster automation = Better Efficiency

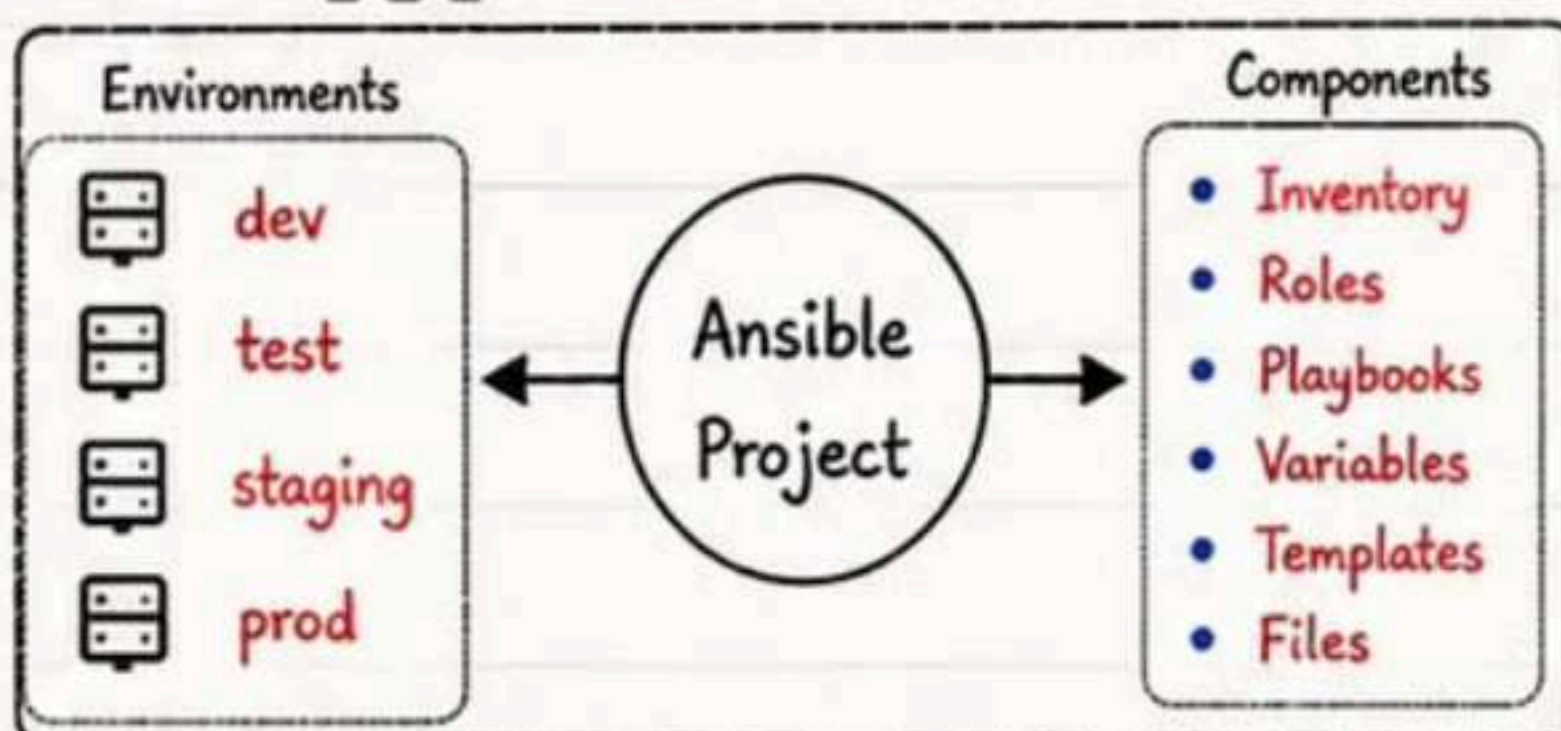
⑥ Reusable Playbooks

- Use **Roles** for reusability.
- Use **variables** for flexibility.
- Use **includes/imports** for modular playbooks.
- Use **templates** for dynamic files.

✓ Write once, use everywhere!

⑦ Project Organization

- Keep everything organized.
- Separate environments.
- Use meaningful directories.
- Maintain **README.md** for documentation.



⚠ Common Mistakes

- ✗ Not using Idempotent modules
- ✗ Hardcoding variables
- ✗ Poor folder structure
- ✗ Ignoring error handling
- ✗ Not testing playbooks

★ Golden Rule

Automate smartly, keep it simple, secure and scalable!

Common Interview Questions

① Q. What is Ansible?

→ Ansible is an open-source Automation tool used for Configuration Management, Application Deployment, Task Automation, and Orchestration. It uses SSH to communicate and does not require any agent on target machines.



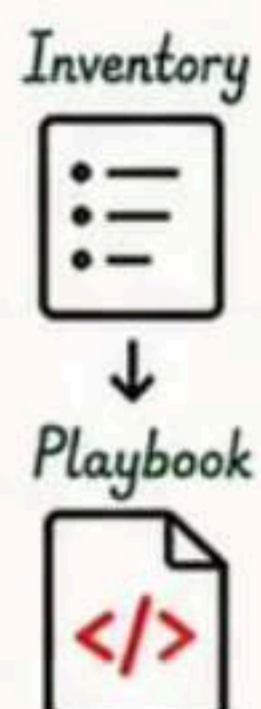
② Q. Why is Ansible Agentless?

→ Ansible uses SSH (or WinRM for Windows) to connect to managed nodes. No software/agent needs to be installed on remote servers, which makes it lightweight, secure and easy to manage.



③ Q. What is the difference between Inventory and Playbook?

→ **Inventory**: A file that lists all the Hosts and Groups on which Ansible will run.
→ **Playbook**: A YAML file that defines Tasks, Variables, Roles and Handlers to be executed on those hosts.



④ Q. What is the difference between Module and Role?

→ **Module**: A single unit of code that performs a specific task (e.g., copy, service, user).
→ **Role**: A way to organize playbooks and related files (tasks, vars, templates, handlers) in a structured directory.



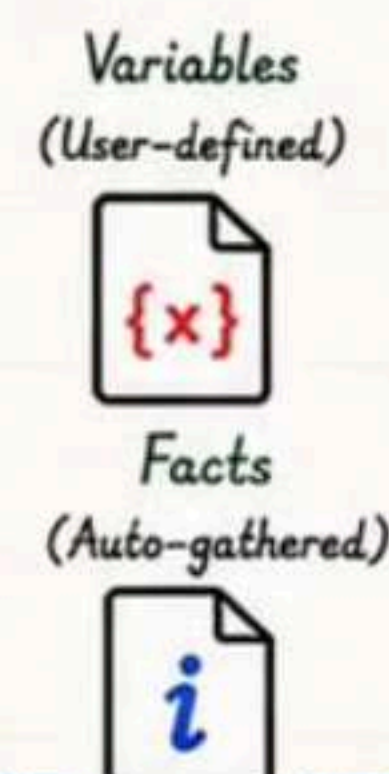
⑤ Q. Difference between command and shell?

→ **command**: Executes commands directly without using a shell. Safer and faster. No shell features like pipes, redirection, wildcards.
→ **shell**: Executes commands through the shell. Supports shell features like pipes (|), redirects (>) and environment variables.



⑥ Q. Difference between Variables and Facts?

→ **Variables**: User-defined or system-defined values used in Playbooks.
→ **Facts**: Information automatically gathered about managed nodes using gather_facts (hardware, OS, IP, etc.). Stored in ansible_facts.



⑦ Q. What is Ansible Vault?

→ Ansible Vault is used to encrypt sensitive data such as passwords, API keys, tokens and secret variables in files. It ensures confidential information is not stored in plain text.



⑧ Q. What are Handlers in Ansible?

→ **Handlers** are special tasks that run only when notified by another task. They are typically used to restart services or reload configurations. Handlers run once at the end of the Play.



Quick Comparison

Concept	What is it?	Key Point
Ansible	Automation tool	Agentless, uses SSH
Inventory	Hosts & Groups list	Where to run
Playbook	YAML automation file	What to run
Module	Single task unit	Performs specific action
Role	Structured collection	Reusability & scalability
Variables	User/System defined data	Provide dynamic values
Facts	Auto discovered data	Stored in ansible_facts
Handlers	Tasks that run on notify	Execute once at the end

★ Interview Tip

- ✓ Focus on real-world examples.
- ✓ Understand YAML structure.
- ✓ Know the difference clearly (Inventory vs Playbook, Module vs Role).
- ✓ Hands-on practice is the key!
- ✓ Explain with use cases for better impression.



Remember

Ansible = Simple, Powerful, Agentless Automation.
Master the concepts, and DevOps becomes easier!



Common Mistakes & Troubleshooting

① YAML Indentation Errors



YAML is space-sensitive.

Wrong indentation breaks the Playbook.

❌ Wrong

```
- hosts: web
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
# Wrong indentation!
```

✅ Correct

```
- hosts: web
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
# Correct indentation
```

Tip: Use 2 spaces for indentation (never tabs).

② Inventory Issues



Wrong inventory leads to unreachable hosts or wrong grouping.

Common Problems

- Wrong IP / Hostname
- Host not added to group
- Wrong group name
- Inventory file not loaded
- Duplicate hosts

Solutions

- Verify IP/hostname & connectivity
- Check group & host names
- Use `ansible-inventory --list`
- Use `-i <inventory>` properly
- Keep inventory clean & updated

③ SSH Problems



Ansible uses SSH. Any SSH issue will cause connection failures.

Common Problems

- SSH service not running
- Wrong username
- Wrong/private key
- SSH port blocked
- Host key verification failed

Solutions

- Start SSH service
- Use correct user (`-u user`)
- Provide correct key (`-i key.pem`)
- Open port 22 in firewall
- Run `ssh-keyscan <host>`

④ Permission Errors



Insufficient permissions cause task failures (files, directories, become, etc.).

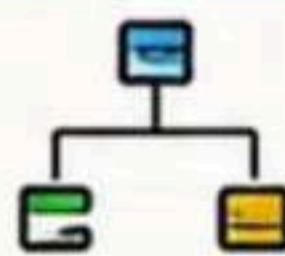
Common Problems

- No sudo/become access
- Insufficient file permissions
- Unable to write to directory
- SSH key permission too open

Solutions

- Use `become: yes`
- Fix file/dir permissions (`chmod/chown`)
- Ensure writable paths
- `chmod 600` on private key

⑤ Variable Precedence Issues



Unexpected values due to precedence order. Hard to find which variable is used.

Problem

Same variable defined in multiple places (defaults, vars, group_vars, host_vars, extra_vars, etc.)
Confusion about final value.

Precedence (high → low)

1. Extra vars (`-e`)
2. Task vars / `set_facts`
3. Block vars
4. Role vars
5. Play vars
6. Inventory (`host_vars`, `group_vars`)
7. Role defaults

⑥ Module Selection Mistakes



Using wrong module can cause failures or non-idempotent behavior.

Mistakes

- Using command instead of module
- Using shell when module exists
- Using apt on RHEL / yum on Debian
- Not using idempotent modules

Best Practice

- Prefer built-in modules
- Modules are idempotent
- Choose OS specific modules
- Check module docs (`ansible-doc`)

⑦ General Troubleshooting Tips



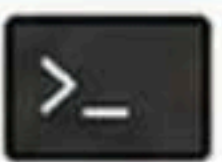
- ✓ Run playbook in verbose mode: `ansible-playbook -v` or `-vvv`
- ✓ Syntax check: `ansible-playbook --syntax-check <file.yml>`
- ✓ Check inventory: `ansible-inventory --list`
- ✓ Ping test: `ansible all -m ping`
- ✓ Limit hosts: `--limit <host/group>`
- ✓ Check facts: `ansible <host> -m setup`
- ✓ Read logs and error carefully
- ✓ Use `--diff` to see changes before applying
- ✓ Keep Playbooks small, modular & idempotent



Interview Tip

Always read the error message carefully. 95% of issues are solved by checking logs, inventory, permissions and indentation.

⑧ Quick Troubleshooting Commands



Purpose	Command
Check connectivity	<code>ansible all -m ping</code>
Inventory list	<code>ansible-inventory --list</code>
Syntax check	<code>ansible-playbook --syntax-check site.yml</code>
Verbose output	<code>ansible-playbook -vvv site.yml</code>
Limit execution	<code>ansible-playbook site.yml --limit web</code>
Check module docs	<code>ansible-doc <module_name></code>
Check facts	<code>ansible <host> -m setup</code>



Remember

Ansible works best when you follow best practices: small, simple, idempotent and reusable Playbooks.

Quick Revision Cheat Sheet

① Most Used Commands

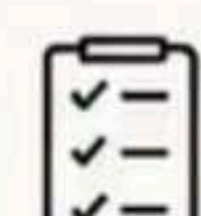
Command	Purpose
<code>ansible --version</code>	Check Ansible version
<code>ansible all -m ping</code>	Check connectivity to all hosts
<code>ansible all -a "uptime"</code>	Run ad-hoc command
<code>ansible-playbook playbook.yml</code>	Run a Playbook
<code>ansible-inventory --list</code>	List inventory in JSON
<code>ansible-galaxy role install <role></code>	Install Role from Galaxy
<code>ansible-vault encrypt <file></code>	Encrypt a file with Vault

② Playbook Structure (YAML)

```

---
- name: Playbook Name
  hosts: web
  become: yes
  vars:
    key: value
  tasks:
    - name: Task name
      module_name:
        option: value
  handlers:
    - name: Restart service
      service:
        name: httpd
        state: restarted
  
```

 Playbook level settings

 List of Tasks

 Triggered at the end

③ Inventory Syntax

INI Format (default)

```

[web]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[db]
db1 ansible_host=192.168.1.20

[web:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_rsa
  
```

Simple & Effective

YAML Format

```

all:
- children:
  - web:
    - hosts:
      - web1:
        ansible_host: 192.168.1.10
      - web2:
        ansible_host: 192.168.1.11
  - db:
    - hosts:
      - db1:
        ansible_host: 192.168.1.20
  
```

Great for Large Environments

④ Module Summary (Commonly Used)

Category	Module	Purpose
 System	yum, apt, dnf	Manage packages
 Service	service, systemd	Manage services
 Files	copy, template, file	Manage files & templates
 User	user, group	Manage users & groups
 Network	uri, get_url	Handle URLs & APIs
 Archive	unarchive, archive	Extract / Create archives

⑥ Interview Revision Table

Concept	Key Point
Ansible	Agentless, uses SSH, YAML based
Inventory	Hosts & Groups definition
Playbook	YAML file with tasks & handlers
Module	Single unit of code to perform task
Role	Organize playbooks & tasks
Variables	Store data & make dynamic
Facts	Auto discovered system info
Handlers	Run only when notified
Vault	Encrypt sensitive data
Tags	Run specific tasks
Idempotency	Same result, no unwanted changes

⑤ Vault Commands

```

ansible-vault encrypt <file> → Encrypt a file
ansible-vault view <file> → View encrypted file
ansible-vault edit <file> → Edit encrypted file
ansible-vault decrypt <file> → Decrypt a file
ansible-vault rekey <file> → Change vault password
  
```



⑦ Quick Playbook Example

```

---
- name: Install & Start Nginx
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present
    - name: Start Nginx
      service:
        name: nginx
        state: started
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
  
```

Simple Playbook with Tasks & Handler

⑧ Key Takeaways

- ✓ Ansible = Simple, Powerful, Agentless Automation
- ✓ Write Playbooks in YAML
- ✓ Use Inventory to manage Hosts & Groups
- ✓ Modules are the building blocks
- ✓ Roles help in reusability & organization
- ✓ Use Variables, Facts & Templates for dynamic automation
- ✓ Vault protects your sensitive data
- ✓ Handlers keep services in sync
- ✓ Idempotency ensures consistent & reliable runs
- ✓ Follow best practices for clean & maintainable code

Remember!

Automation is not about writing complex code, it's about writing smart, reusable & reliable Playbooks!

★ Pro Tip: Keep Playbooks small, modular & idempotent. Test in lower environment before Production!



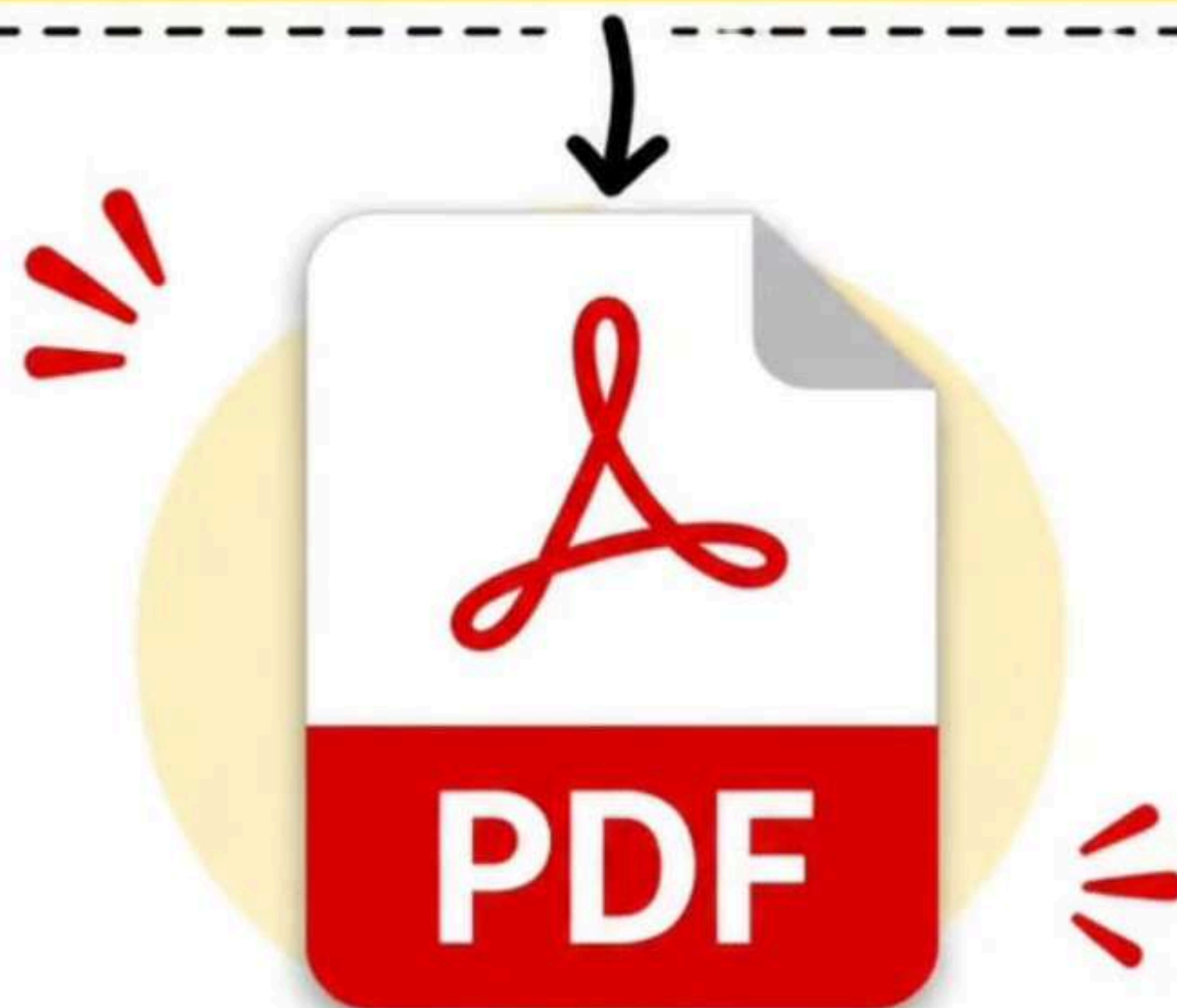
**QA
INSIGHTS**

— QUALITY INSIGHTS, BETTER TESTING! —

FOLLOW & COMMENT

— TO GET —

THE PDF



----- **@qa_insights** -----

TESTING INSIGHTS | TOOLS | TUTORIALS | CAREER TIPS | RESOURCES